

Week 2

Variables and Input

Store, protect, validate, and use script data

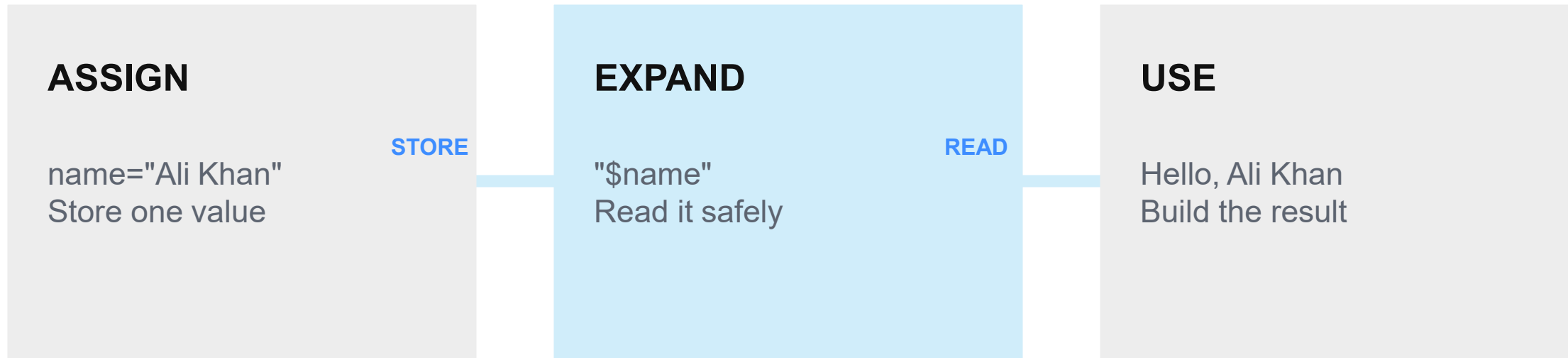
Two 90-minute lessons | Milestone M2: Input Handler

Instructor: Muhammad Khalid Khan

By the end of Week 2, you can control script input

- 01 Create and expand variables correctly
- 02 Compare unquoted, single-quoted, and double-quoted text
- 03 Capture command output with \$(command)
- 04 Accept interactive input with read -r
- 05 Use \$0, \$1, \$#, and quoted \$@
- 06** Validate arguments and preserve spaces

Variables give reusable names to script data



Assignment stores data. Expansion retrieves it. Quoting protects the value's boundaries.

Reliable input follows a four-stage path



```
input -> validate -> variable -> quoted expansion -> output + exit  
status
```

Four patterns make variable expansion predictable

```
course_name="Bash Scripting"
student_name="Ali Khan"
today="$(date +%F) "

printf 'Student: %s\n' "$student_name"
printf 'File: %s\n'
"${student_name}_report"
```

01 ASSIGNMENT

No spaces around the equals sign.

02 EXPANSION

Double quotes preserve one value.

03 BRACES

Clarify where the name ends.

04 SUBSTITUTION

\$(command) captures output.

Single and double quotes preserve different meanings

SINGLE QUOTES

' \$name '

Everything remains literal.
Variables do not expand.
Commands are not substituted.

DOUBLE QUOTES

" \$name "

Variables expand.
Spaces remain one value.
Wildcards remain protected.

Default safety: double quote variable expansions unless splitting or globbing is intentional.

Guided demo: build a personalized server greeting

```
#!/usr/bin/env bash

read -r -p "Enter your full name: " student_name

if [[ -z "$student_name" ]]; then
    printf '%s\n' 'Name is required' >&2
    exit 1
fi

current_user="$(whoami) "
printf 'Hello %s from %s\n' "$student_name"
"$current_user"
```

TEST

```
bash server-greeting.sh
```

Input: Ali Khan

Input: Lab\User

Input: <empty>

CHECK

Spaces stay intact. Empty input fails safely.

Interactive input must be read, validated, and quoted

READ

Accept the user's
complete input

```
read -r name
```

VALIDATE

Reject missing or
invalid data

```
[[ -z "$name" ]]
```

USE

Preserve the value
as one argument

```
printf '%s' "$name"
```

A prompt does not make input trustworthy - validation does.

Arguments make scripts reusable and non-interactive

POSITIONAL PARAMETERS

`$0` `$1` `$2` `$#`

`$0` = script name
`$1` and `$2` = arguments
`$#` = argument count

ALL ARGUMENTS

`"$@"`

Every argument is preserved
including values with spaces

```
./argument-demo.sh "Linux Admin" WSL AWS
```

A safe file inspector validates before using \$1

```
#!/usr/bin/env bash

if [[ $# -ne 1 ]]; then
    printf 'Usage: %s <path>\n' "$0" >&2
    exit 2
fi

target_path="$1"

if [[ ! -e "$target_path" ]]; then
    printf 'Missing: %s\n' "$target_path" >&2
    exit 1
fi
```

VALIDATION ORDER

- 01 Check argument count
- 02 Display usage
- 03 Store quoted \$1
- 04 Check the path
- 05 Report and exit

Critical test: a path containing spaces must remain one value.

Two labs turn data handling into evidence

LAB 1

Server Greeting

- Prompt with read -r
- Reject empty input
- Use command substitution
- Preserve a full name
- Explain each expansion

LAB 2

Safe File Inspector

- Require one path argument
- Display usage on error
- Quote \$1 every time
- Test a path with spaces
- Use meaningful exit codes

Required proof: normal input + spaces + empty input + missing path + exit status

Milestone M2: accept, validate, and preserve input

INPUT HANDLER

You are ready when your script accepts data, rejects bad input, preserves spaces, and explains each expansion.

01

STORE

Correct assignment + command substitution

02

PROTECT

Double quotes preserve boundaries

03

VALIDATE

Usage + failure path + exit code

NEXT: Week 3 adds tests, exit-status decisions, if/elif/else, and case.